Functions & Methods — Input, Output, and Actual Code

Seventeen functions from the supplied Windows 11 LangChain PDF Q&A source. Each entry includes its real Python definition, inputs, outputs, purpose, and a call example. No model results are invented.

Execution note: a method signature and code example are not proof of successful model inference. Run the notebook on the configured PC for real outputs.

01 / 17 — list_pdfs()

File: src/pdf_reader.py

Purpose: Search directory for PDF filenames.

Input: folder: Path

Output: list[Path]

Example invocation: `list_pdfs(PDF_DIR)`

Original function body (from the ZIP):

```
def list_pdfs(folder: Path) -> list[Path]:
    """Input: thu muc. Output: danh sach tep PDF, sap xep theo ten."""
    return sorted(
        (p for p in folder.iterdir() if p.is_file() and p.suffix.lower() == '.pdf'),
        key=lambda p: p.name.lower(),
    ) if folder.exists() else []
```

02 / 17 — load_pdfs()

File: src/pdf_reader.py

Purpose: Extract text pages and retain source metadata.

Input: folder: Path

Output: list[Document]

Example invocation: `load_pdfs(PDF_DIR)`

Original function body (from the ZIP):

```
def load_pdfs(folder: Path) -> list[Document]:
    """Input: thu muc chua PDF. Output: trang co van ban duoi dang Document."""
    pdfs = list_pdfs(folder)
    if not pdfs:
        raise FileNotFoundError(f'Chua co file PDF trong: {folder}')

    documents: list[Document] = []
    for pdf in pdfs:
        try:
            pages = PyPDFLoader(str(pdf)).load()
        except Exception as exc:
            raise RuntimeError(f'Khong doc duoc PDF {pdf.name}: {exc}') from exc
        for page in pages:
            if page.page_content and page.page_content.strip():
                # 'page' tu PyPDFLoader bat dau bang 0 (trang thu 1 -> 0).
                page.metadata['source'] = pdf.name
                documents.append(page)
    if not documents:
        raise ValueError('PDF khong trich xuat duoc chu. Neu la ban scan anh, can OCR.')
    return documents
```

03 / 17 — split_documents()

File: src/text_chunker.py

Purpose: Split pages and attach chunk IDs.

Input: documents, chunk_size, chunk_overlap

Output: list[Document]

Example invocation: `split_documents(pages,900,150)`

Original function body (from the ZIP):

```
def split_documents(
    documents: list[Document], chunk_size: int = 900, chunk_overlap: int = 150
) -> list[Document]:
    """Input: documents + do dai/overlap. Output: danh sach chunk Document."""
    if chunk_size <= 0:
        raise ValueError('chunk_size phai lon hon 0')
    if not (0 <= chunk_overlap < chunk_size):
        raise ValueError('chunk_overlap phai tu 0 den nho hon chunk_size')
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=['\n\n', '\n', '. ', ' ', ''],
    )
    chunks = splitter.split_documents(documents)
    for idx, doc in enumerate(chunks, start=1):
        doc.metadata['chunk_id'] = idx
    return chunks
```

04 / 17 — make_embeddings()

File: src/vector_store.py

Purpose: Create multilingual text embedding model.

Input: model_name: str

Output: HuggingFaceEmbeddings

Example invocation: `make_embeddings(EMBEDDING_MODEL)`

Original function body (from the ZIP):

```
def make_embeddings(model_name: str) -> HuggingFaceEmbeddings:
    """Input: ten model HuggingFace. Output: bo ma hoa van ban thanh vector."""
    return HuggingFaceEmbeddings(model_name=model_name)
```

05 / 17 — build_vector_store()

File: src/vector_store.py

Purpose: Create vector index using chunks.

Input: documents, embeddings

Output: FAISS

Example invocation: `build_vector_store(chunks,emb)`

Original function body (from the ZIP):

```
def build_vector_store(documents: list[Document], embeddings: HuggingFaceEmbeddings) -> FAISS:
    """Input: chunks + embedding. Output: FAISS vector store trong RAM."""
    if not documents:
        raise ValueError('Khong co doan van ban de tao FAISS.')
    return FAISS.from_documents(documents, embeddings)
```

06 / 17 — save_vector_store()

File: src/vector_store.py

Purpose: Persist FAISS index locally.

Input: store, folder

Output: None; writes index files

Example invocation: `save_vector_store(store,INDEX_DIR)`

Original function body (from the ZIP):

```
def save_vector_store(store: FAISS, folder: Path) -> None:
    """Input: FAISS + thu muc. Output: tep index.faiss va index.pkl tren o dia."""
    folder.mkdir(parents=True, exist_ok=True)
    store.save_local(str(folder))
```

07 / 17 — load_vector_store()

File: src/vector_store.py

Purpose: Load trusted local index only.

Input: folder, embeddings

Output: FAISS

Example invocation: `load_vector_store(INDEX_DIR,emb)`

Original function body (from the ZIP):

```
def load_vector_store(folder: Path, embeddings: HuggingFaceEmbeddings) -> FAISS:
    """Input: index da tao + embedding. Output: FAISS vector store de tim kiem.

    CANH BAO: index.pkl dung pickle; chi tai lai index tin cay do ban tao.
    Khong copy index.pkl tu nguon la vao folder nay.
    """
    if not (folder / 'index.faiss').exists() or not (folder / 'index.pkl').exists():
        raise FileNotFoundError('Chua tao index FAISS')
    return FAISS.load_local(str(folder), embeddings, allow_dangerous_deserialization=True)
```

08 / 17 — retrieve_documents()

File: src/qa.py

Purpose: Find at most k relevant chunks.

Input: store, question, k

Output: list[Document]

Example invocation: `retrieve_documents(store,question,4)`

Original function body (from the ZIP):

```
def retrieve_documents(store, question: str, k: int = 4) -> list[Document]:
    """Input: FAISS + cau hoi + top_k. Output: toi da k doan lien quan."""
    if not question.strip():
        return []
    return store.as_retriever(search_kwargs={'k': k}).invoke(question)
```

09 / 17 — format_context()

File: src/qa.py

Purpose: Annotate each retrieved passage with file and page.

Input: documents: list[Document]

Output: str

Example invocation: `format_context(retrieved)`

Original function body (from the ZIP):

```
def format_context(documents: list[Document]) -> str:
    """Input: cac chunk tim duoc. Output: chuoi ngu canh co [i], ten tep, trang."""
    parts: list[str] = []
    for idx, doc in enumerate(documents, start=1):
        page_num = int(doc.metadata.get('page', 0)) + 1
        filename = doc.metadata.get('source', 'khong-ro.pdf')
        parts.append(f'[{idx}] File: {filename}; Trang: {page_num}\n{doc.page_content}')
    return '\n\n'.join(parts)
```

10 / 17 — generate_answer()

File: src/qa.py

Purpose: Run ChatOllama via LCEL and return text.

Input: question, documents, model_name

Output: str

Example invocation: `generate_answer(question,retrieved,LLM_MODEL)`

Original function body (from the ZIP):

```
def generate_answer(question: str, documents: list[Document], model_name: str) -> str:
    """Input: cau hoi, chunk truy xuat, model Ollama. Output: cau tra loi String."""
    if not documents:
        return 'Tôi không tìm th□y thông tin này trong PDF.'
    llm = ChatOllama(model=model_name, temperature=0)
    chain = PROMPT | llm | StrOutputParser()
    return chain.invoke({'question': question, 'context': format_context(documents)}).strip()
```

11 / 17 — answer_question()

File: src/qa.py

Purpose: Return answer plus retrieved source metadata.

Input: store, question, model_name, k

Output: dict: answer, sources, retrieved_count

Example invocation: `answer_question(store,question,LLM_MODEL,k=4)`

Original function body (from the ZIP):

```
def answer_question(store, question: str, model_name: str, k: int = 4) -> dict:
    """Input: FAISS, cau hoi, model, k. Output: dict(answer, sources, retrieved_count)."""
    docs = retrieve_documents(store, question, k=k)
    answer = generate_answer(question, docs, model_name)
    sources = [
        {'file': doc.metadata.get('source', '?'),
         'page': int(doc.metadata.get('page', 0)) + 1,
         'chunk_id': doc.metadata.get('chunk_id')}
        for doc in docs
    ]
    return {'answer': answer, 'sources': sources, 'retrieved_count': len(docs)}
```

12 / 17 — pdf_signature()

File: src/pipeline.py

Purpose: Read PDF file names, sizes, and timestamps.

Input: pdf_dir: Path

Output: list[dict]

Example invocation: `pdf_signature(PDF_DIR)`

Original function body (from the ZIP):

```
def pdf_signature(pdf_dir: Path = PDF_DIR) -> list[dict]:
    """Input: thu muc PDF. Output: thong tin file de nhan biet thay doi."""
    return [
        {'name': p.name, 'size': p.stat().st_size, 'mtime_ns': p.stat().st_mtime_ns}
        for p in list_pdfs(pdf_dir)
    ]
```

13 / 17 — build_manifest()

File: src/pipeline.py

Purpose: Combine file signature and processing settings.

Input: pdf_dir: Path

Output: dict

Example invocation: `build_manifest(PDF_DIR)`

Original function body (from the ZIP):

```
def build_manifest(pdf_dir: Path = PDF_DIR) -> dict:
    """Input: thu muc PDF. Output: metadata cua index va tham so."""
    return {
        'pdfs': pdf_signature(pdf_dir),
        'chunk_size': CHUNK_SIZE,
        'chunk_overlap': CHUNK_OVERLAP,
        'embedding_model': EMBEDDING_MODEL,
    }
```

14 / 17 — prepare_store()

File: src/pipeline.py

Purpose: Reuse or rebuild cached index as required.

Input: pdf_dir, index_dir

Output: tuple: FAISS, created, pages, chunks

Example invocation: `prepare_store(PDF_DIR,INDEX_DIR)`

Original function body (from the ZIP):

```
def prepare_store(pdf_dir: Path = PDF_DIR, index_dir: Path = INDEX_DIR):
    """Input: thu muc PDF/index. Output: tuple(FAISS, created_new, num_pages, num_chunks)."""
    current = build_manifest(pdf_dir)
    if not current['pdfs']:
        raise FileNotFoundError(f'Chua co PDF trong thu muc: {pdf_dir}')
    manifest_path = index_dir / 'manifest.json'
    is_cached = False
    if manifest_path.exists():
        try:
            is_cached = (json.loads(manifest_path.read_text(encoding='utf-8')) == current
                         and (index_dir / 'index.faiss').exists()
                         and (index_dir / 'index.pkl').exists())
        except (OSError, ValueError):
            pass
    embeddings = make_embeddings(EMBEDDING_MODEL)
    if is_cached:
        print('[INFO] PDF khong doi - su dung index FAISS da luu.')
        try:
            return load_vector_store(index_dir, embeddings), False, None, None
        except Exception as exc:
            print(f'[WARN] Index cu khong doc duoc ({exc}). Dang tao moi...')
    pages = load_pdfs(pdf_dir)
    chunks = split_documents(pages, CHUNK_SIZE, CHUNK_OVERLAP)
    print(f'[INFO] Da doc {len(pages)} trang co chu, tao {len(chunks)} chunks.')
    store = build_vector_store(chunks, embeddings)
    save_vector_store(store, index_dir)
    manifest_path.write_text(json.dumps(current, ensure_ascii=False, indent=2), encoding='utf-8')
    return store, True, len(pages), len(chunks)
```

15 / 17 — check_ollama()

File: app.py

Purpose: Check local Ollama service and requested model.

Input: model_name: str

Output: None or RuntimeError

Example invocation: `check_ollama(LLM_MODEL)`

Original function body (from the ZIP):

```
def check_ollama(model_name: str = LLM_MODEL) -> None:
    """Input: model name. Output: None; raise RuntimeError neu Ollama chua san sang."""
    try:
        with urllib.request.urlopen(f'{OLLAMA_URL}/api/tags', timeout=4) as response:
            info = json.load(response)
    except (urllib.error.URLError, TimeoutError, ValueError) as exc:
        raise RuntimeError(
            'Khong ket noi duoc Ollama. Hay mo ung dung Ollama tren Windows; '
            'neu can, mo CMD khac va chay: ollama serve'
        ) from exc
    names = [m.get('name', '') for m in info.get('models', [])]
    if model_name not in names:
        raise RuntimeError(
            f'Chua co model {model_name}. Mo CMD va chay: ollama pull {model_name}'
        )
```

16 / 17 — save_history()

File: app.py

Purpose: Append response, sources and elapsed time to file.

Input: question, result, elapsed

Output: None; appends JSONL

Example invocation: `save_history(question,result,elapsed)`

Original function body (from the ZIP):

```
def save_history(question: str, result: dict, elapsed: float) -> None:
    """Input: cau hoi, cau tra loi, thoi gian. Output: them 1 dong JSONL vao ket_qua."""
    RESULTS_DIR.mkdir(exist_ok=True, parents=True)
    record = {
        'timestamp': datetime.now().isoformat(timespec='seconds'),
        'question': question,
        'answer': result['answer'],
        'sources': result['sources'],
        'elapsed_seconds': round(elapsed, 2),
    }
    with (RESULTS_DIR / 'lich_su_hoi_dap.jsonl').open('a', encoding='utf-8') as stream:
        stream.write(json.dumps(record, ensure_ascii=False) + '\n')
```

17 / 17 — main()

File: app.py

Purpose: Interactive terminal with question and exit loop.

Input: stdin / console input

Output: int exit code

Example invocation: `main()`

Original function body (from the ZIP):

```
def main() -> int:
    """Input: ban phim. Output: in cau tra loi + nguon; 0 neu thoat binh thuong."""
    print('=' * 67)
    print('         DEMO HOI DAP PDF - LANGCHAIN + RAG + OLLAMA')
    print('=' * 67)
    pdfs = list_pdfs(PDF_DIR)
    if not pdfs:
        print(f'[LOI] Hay copy it nhat 1 file .pdf vao: {PDF_DIR}')
        return 1
    print('PDF tim thay: ' + ', '.join(p.name for p in pdfs))
    try:
        check_ollama()
        store, created, pages, chunks = prepare_store()
        print(f'[OK] FAISS: {"vua tao moi" if created else "da tai lai"}.')
        print('Nhap cau hoi bang tieng Viet; nhap exit de thoat.')
        while True:
            try:
                question = input('\nBAN HOI > ').strip()
            except (EOFError, KeyboardInterrupt):
                print('\nKet thuc demo.')
                return 0
            if question.lower() in ('exit', 'quit', 'q', 'thoat'):
                print('Da thoat.')
                return 0
            if not question:
                print('Hay nhap mot cau hoi.')
                continue
            t0 = time.perf_counter()
            try:
                result = answer_question(store, question, LLM_MODEL, TOP_K)
            except Exception as exc:
                print(f'[LOI] Khong the tao cau tra loi: {exc}')
                print('Kiem tra Ollama dang chay va may du RAM.')
                continue
            elapsed = time.perf_counter() - t0

            print('\nTRA LOI:\n' + result['answer'])
            print('\nCAC DOAN DA TRUY XUAT (can kiem chung voi PDF):')
            for source in result['sources']:
                print(f" - {source['file']} | trang {source['page']} | chunk
    {source['chunk_id']}")
            print(f'Thoi gian: {elapsed:.2f} giay')
            save_history(question, result, elapsed)
    except (FileNotFoundError, ValueError, RuntimeError) as exc:
        print(f'\n[LOI] {exc}')
        return 1
    except Exception as exc:
        print(f'\n[LOI KHONG MONG DOI] {type(exc).__name__}: {exc}')
        return 1
```